

Packet Finite-State Machine

Implementation of RISC-V extensions for Control Flow Integrity challenge

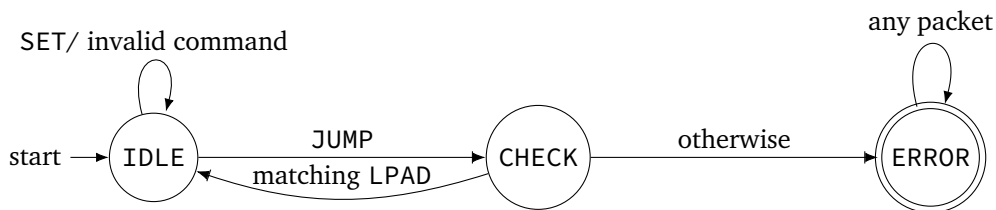
Overview

This submission implements a three-state SystemVerilog finite-state machine that accepts one 32-bit packet per clock cycle. The command is in `packet[31:24]` and the data is in `packet[23:0]`.

Command	Value	Meaning
SET	8'h01	Store data while in IDLE
JUMP	8'h02	Move from IDLE to CHECK
LPAD	8'h03	Match the stored data in CHECK

Reset is synchronous and active-high. It initializes the FSM to IDLE. A matching LPAD returns from CHECK to IDLE; any other packet in CHECK enters permanent ERROR.

State Transitions



In IDLE, SET and other non-JUMP packets leave the FSM in IDLE; SET updates the internal label register.

Implementation and Verification

The FSM is implemented in `packet_fsm.sv`. Since state and label are updated only on clock edges, the control is expressed in one sequential `always_ff` block, instead of using conventional two-block design. Using `always_ff` ensures that the nature of the circuit is sequential and the tool can warn if it infers any unintentional combinational behavior. The cocotb testbench, `test_packet_fsm.py`, verifies the public packet-to-state behavior as a black box, including reset, valid transitions, rejected packets, label replacement, and the absorbing error state.

Run the tests with:

```
make SIM=verilator
```

If you want to use Icarus verilog for running the tests, change the above command to.

```
make SIM=icarus
```

Submission Link

The submission is available on [GitHub](#).